Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №1
на тему
«СИММЕТРИЧНАЯ КРИПТОГРАФИЯ. СТАНДАРТ ШИФРОВАНИЯ ГОСТ 28147-89»
БГУИР 6-05-0612-02 21

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ПОЛЯКОВ Кирилл Борисович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Теоретические сведения 4
2.1 Стандарт ГОСТ 28147-89 4
2.2 Функция раунда 4
2.3 Расписание ключей 4
2.4 Режим простой замены (*ECB*) 5
2.5 Режим гаммирования 5
2.6 Режим гаммирования с обратной связью 5
2.7 Режим выработки имитовставки (*MAC*) 6
3 Выполнение работы 7
3.1 Архитектура реализации 7
3.2 Реализация блочного шифра 7
3.3 Реализация режима простой замены 8
3.4 Реализация режима гаммирования 8
3.5 Реализация режима гаммирования с обратной связью 8
3.6 Реализация вычисления кода аутентичности 9
Заключение 10
Список использованных источников 11
Приложение А (Обязательное) Листинг программного кода 12

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является изучение стандарта шифрования ГОСТ 28147-89 и получение практических навыков работы с блочными шифрами путём самостоятельной реализации алгоритма шифрования и четырёх режимов его применения: простой замены (*ECB*), гаммирования (*CTR*), гаммирования с обратной связью (*CFB*) и вычисления кода аутентичности (*MAC*).

Для достижения поставленной цели необходимо решить следующие задачи:

1 Изучить теоретические основы стандарта ГОСТ 28147-89: структуру блочного шифра, функцию раунда, расписание ключей.

2 Реализовать базовый блочный шифр с поддержкой 32 раундов и таблицы подстановок (*S-Box*).

3 Реализовать четыре режима работы: простой замены, гаммирования, гаммирования с обратной связью и вычисления кода аутентичности.

4 Провести тестирование корректности работы всех режимов посредством последовательного шифрования и дешифрования.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Стандарт ГОСТ 28147-89

ГОСТ 28147-89 – советский и российский стандарт симметричного блочного шифрования, принятый в 1989 году [1]. Алгоритм является основой для ряда более поздних стандартов, включая ГОСТ Р 34.12-2015 «Магма» [2].

Основные параметры стандарта:

– Размер блока: 64 бита (8 байт)

– Размер ключа: 256 бит (32 байта)

– Количество раундов: 32

– Размер подключа: 32 бита

Алгоритм основан на сети Фейстеля и использует набор из восьми таблиц подстановок (*S-Box*), каждая из которых преобразует 4-битный вход в 4-битный выход. Ключ разбивается на восемь 32-битных подключей $K_1, K_2, \ldots, K_8$, которые в циклическом порядке применяются к раундам шифрования.

Конкретный вид таблиц подстановок стандарт не задаёт: блок подстановки объявлен долговременным ключевым элементом, поставляемым отдельно, а требование к нему сводится к тому, чтобы каждый из восьми узлов был перестановкой чисел от 0 до 15. В реализации используется набор $\pi_1, \ldots, \pi_8$ из параметра *id-tc26-gost-28147-param-Z* [3]

2.2 Функция раунда

Функция раунда *f* определяет преобразование правой половины блока в сети Фейстеля. Для входных значений L (левая половина, 32 бита) и подключа K_i (32 бита) вычисление выполняется следующим образом:

– Вычисляется сумма $S = (L + K_i) \bmod 2^{32}$.

– Значение S разбивается на восемь 4-битных частей $s_0, s_1, \ldots, s_7$.

– Каждая часть s_j заменяется соответствующим значением из таблицы подстановок π_j: $s_{j'} = \pi_{j(s_j)}$.

– Результаты конкатенируются в 32-битное слово.

– Выполняется циклический сдвиг влево на 11 позиций.

Конечный результат является выходом функции раунда.

2.3 Расписание ключей

Для шифрования и дешифрования используются 32 подключа, формируемые из 256-битного ключа:

– Ключ разделяется на восемь 32-битных слов $K_1, K_2, \ldots, K_8$.

– При шифровании в первых 24 раундах подключи применяются циклически: $k_i = K_{(i \bmod 8)+1}$, а в последних восьми раундах порядок обращается: $k_{25} = K_8, k_{26} = K_7, \ldots, k_{32} = K_1$.

– При дешифровании первые восемь подключей используются в прямом порядке ($k_1 = K_1, \ldots, k_8 = K_8$), а оставшиеся 24 обращаются циклически ($k_9 = K_8, \ldots, k_{16} = K_1, \ldots, k_{32} = K_1$).

Такая схема обеспечивает различие в порядке применения ключей при шифровании и дешифровании, что упрощает аппаратную реализацию.

2.4 Режим простой замены (*ECB*)

Режим простой замены является наиболее простым режимом работы блочного шифра. [1] Исходный текст разбивается на блоки по 8 байт, каждый блок шифруется независимо остальных с использованием одного и того же ключа. Последний блок дополняется по стандарту *PKCS#7*: если длина данных не кратна размеру блока, добавляются байты со значением, равным числу добавленных байт.

Преимуществом режима является возможность параллельной обработки блоков. Основным недостатком является сохранение статистических закономерностей исходного текста в шифротексте: одинаковые блоки входных данных всегда приводят к одинаковым блокам шифротекста.

2.5 Режим гаммирования

В режиме гаммирования шифрование выполняется побитовой операцией *XOR* между исходными данными и псевдослучайной последовательностью – гаммой. Гамма генерируется на основе начального вектора (*IV*) длиной 8 байт и ключа шифрования.

Процесс генерации гаммы:

1 Начальный вектор шифруется 32-раундовым блочным шифром для получения начальных значений счётчиков n_3 и n_4.

2 На каждой итерации счётчики инкрементируются: n_3 увеличивается на $0\text{x}1010101\text{B} \bmod 2^{32}$, n_4 – на $0\text{x}10101041 \bmod (2^{32} - 1)$.

3 Обновлённые значения счётчиков объединяются в блок и шифруются для получения следующего сегмента гаммы.

Шифротекст вычисляется как $C_i = M_i \oplus \Gamma_i$, где M_i – очередной байт исходных данных, Γ_i – соответствующий байт гаммы. Дешифрование выполняется аналогично: $M_i = C_i \oplus \Gamma_i$. Начальный вектор передаётся вместе с шифротекстом и не является секретным.

2.6 Режим гаммирования с обратной связью

Режим гаммирования с обратной связью объединяет элементы режимов простой замены и гаммирования. [1] Каждый блок шифротекста зависит не только от текущего блока открытого текста, но и от предыдущего блока шифротекста.

Процесс шифрования:

1 Данные дополняются по стандарту *PKCS#7*.

2 В качестве начального *IV* используется случайный 8-байтовый вектор.

3 Для каждого блока генерируется гамма на основе предыдущего блока шифротекста (или *IV* для первого блока).

4 Текущий блок открытого текста складывается по модулю 2 с полученной гаммой.

Дешифрование выполняется аналогично: гамма генерируется на основе предыдущего блока шифротекста, после чего текущий блок шифротекста складывается по модулю 2 с гаммой для получения открытого текста.

2.7 Режим выработки имитовставки (*MAC*)

Имитовставка (*MAC*) в стандарте ГОСТ 28147-89 используется для проверки целостности и подлинности сообщений [1]. Алгоритм вычисления *MAC* основан на вычислительной схеме *CBC* с использованием 16-раундового блочного шифра.

Основные особенности:

– Последний блок дополняется нулевыми байтами.

– Длина результирующего *MAC* составляет от 1 до 32 бит.

– Минимальная длина сообщения – два блока (16 байт).

Алгоритм выработки имитовставки:

1 Первый блок открытых данных записывается в 32-разрядные регистры N_1 и N_2 и шифруется первыми 16 циклами в режиме простой замены.

2 К полученному результату побитно по модулю 2 прибавляется следующий блок (последний при необходимости дополняется нулями), после чего сумма снова шифруется 16 циклами.

3 Из результата выбирается имитовставка длиной L бит: с бита номер $32 - L$ до 32.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Архитектура реализации

Реализация состоит из двух основных компонентов: библиотеки криптографических алгоритмов и интерфейса управления.

Библиотека криптографических алгоритмов включает следующие модули:

– Базовый блочный шифр: таблица подстановок, функция раунда, расписание ключей, блочное шифрование и дешифрование.
– Режим простой замены (*ECB*).
– Режим гаммирования (*CTR*).
– Режим гаммирования с обратной связью (*CFB*).
– Вычисление кода аутентичности (*MAC*).
– Вспомогательные функции дополнения блоков.

Общий вид интерфейса управления процессом шифрования приведён на рисунке 3.1.

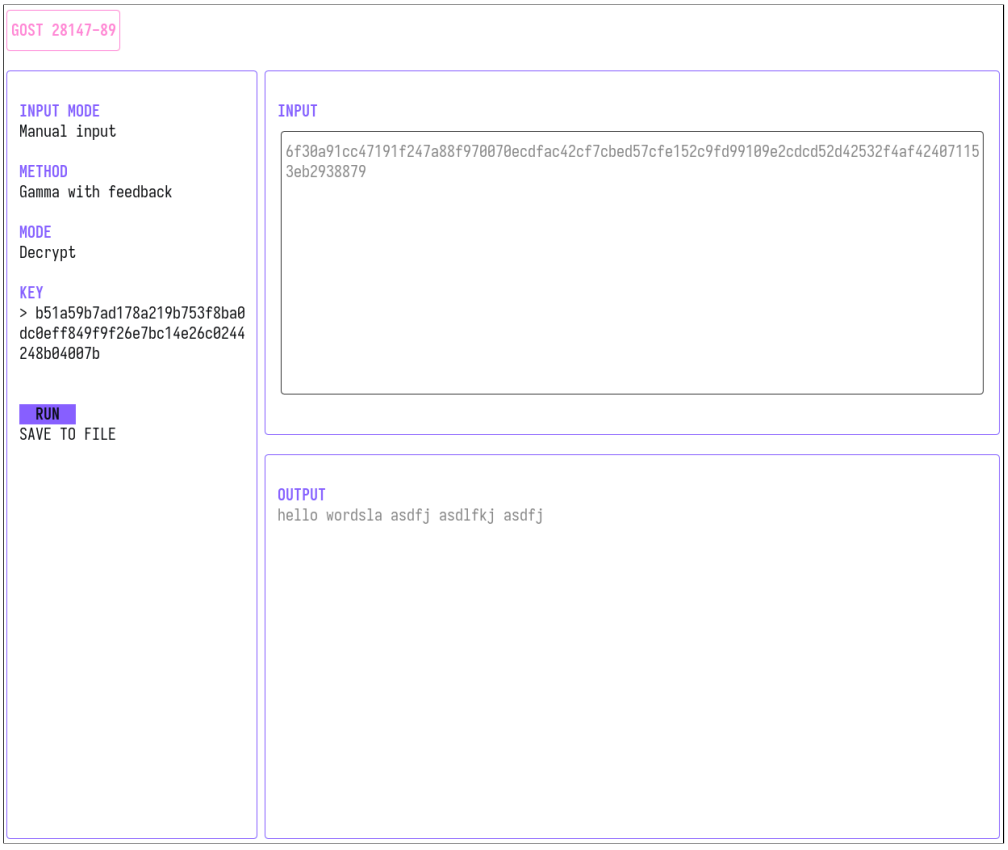


Рисунок 3.1 – Интерфейс управления процессом шифрования

3.2 Реализация блочного шифра

Базовый блочный шифр реализован с использованием таблицы подстановок, содержащей восемь строк по 16 значений каждая. Каждая строка

является перестановкой чисел от 0 до 15, что и требует стандарт. Конкретное наполнение взято из параметра *id-tc26-gost-28147-param-Z* [3]. Функция раунда f выполняет суммирование входа с подключом, замену через таблицу подстановок и циклический сдвиг влево на 11 бит.

Функции *BlockEncryptParts* и *BlockDecryptParts* возвращают левую и правую половину зашифрованного блока как отдельные 32-битные значения, что необходимо для режимов гаммирования и вычисления *MAC*. Функции *BlockEncrypt32* и *BlockDecrypt32* объединяют результат в массив из 8 байт.

Расписание ключей для шифрования применяет подключи в прямом порядке для первых 24 раундов и в обратном для последних 8. При выработке имитовставки используются первые 16 раундов, то есть подключи $K_1, \ldots, K_8$ повторяются дважды и обратный порядок не наступает. Расписание для дешифрования выполняет обратное преобразование.

3.3 Реализация режима простой замены

Режим *ECB* реализован с помощью функций *ECBEncrypt* и *ECBDecrypt*. Функция шифрования дополняет входные данные по стандарту *PKCS#7* и шифрует каждый блок независимо. Функция дешифрования выполняет обратное преобразование и удаляет дополнение.

3.4 Реализация режима гаммирования

Режим гаммирования реализован с помощью функции *Gamma*, которая генерирует последовательность гаммы заданной длины на основевектора и ключа. Счётчики n_3 и n_4 инкрементируются на константы 0x01010101 и 0x01010104 соответственно, что обеспечивает циклический характер гаммы.

Функции *GostGammaEncrypt* и *GostGammaDecrypt* выполняют шифрование и дешифрование с использованием сгенерированной гаммы. Начальный вектор формируется случайным образом и возвращается вместе с шифротекстом.

3.5 Реализация режима гаммирования с обратной связью

Режим гаммирования с обратной связью реализован с помощью функций *GammaWithFeedbackEncrypt* и *GammaWithFeedbackDecrypt*. Функция шифрования дополняет данные по *PKCS#7*, генерирует гамму на основе предыдущего блока шифротекста и выполняет побитовое сложение с текущим блоком. В качестве начального вектора используется случайный 8-байтовый блок.

Функция дешифрования выполняет обратное преобразование, используя предыдущий блок шифротекста для генерации гаммы.

3.6 Реализация вычисления кода аутентичности

Вычисление *MAC* реализовано с помощью функции *MACProduce*, которая принимает данные, ключ и требуемую длину *MAC* (от 1 до 32 бит). Данные дополняются нулевыми байтами до кратности 8. Первый блок шифруется 16 раундами, последующие блоки складываются по модулю 2 с результатом предыдущего шифрования перед обработкой.

Функции *GammaEncryptWithMAC* и *GammaDecryptWithMAC* выполняют шифрование в режиме гаммирования с одновременным вычислением *MAC*. При дешифровании вычисленный *MAC* сравнивается с переданным; в случае несовпадения генерируется паника, свидетельствующая о повреждении данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы реализован блочный шифр ГОСТ 28147-89 с поддержкой четырёх режимов работы: простой замены (*ECB*), гаммирования (*CTR*), гаммирования с обратной связью (*CFB*) и вычисления кода аутентичности (*MAC*).

Реализация базового блочного шифра включает 32 раунда с использованием восьми таблиц подстановок, функцию раунда с суммированием, заменой через *S-Box* и циклическим сдвигом, а также расписание ключей с прямым и обратным порядком применения подключей. Корректность алгоритма подтверждена тестированием: последовательное шифрование и дешифрование полностью восстанавливает исходные данные во всех режимах.

Анализ реализованных режимов показал их различия по области применения. Режим простой замены обеспечивает параллельную обработку, но сохраняет статистические закономерности. Режим гаммирования обеспечивает произвольный доступ к блокам. Режим гаммирования с обратной связью обеспечивает зависимость каждого блока от предыдущих. Вычисление *MAC* гарантирует целостность и подлинность сообщения.

В ходе работы были получены практические навыки работы с блочными шифрами, изучены особенности стандарта ГОСТ 28147-89 и их отличия от западных стандартов, таких как *AES*. Реализованный программный продукт может быть использован для образовательных целей и демонстрации принципов симметричного шифрования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Системы обработки информации. Защита криптографическая. Алгоритм криптографического преобразования : ГОСТ 28147-89. – Введ. 01.07.1990. – М.: Издательство стандартов, 1989. – 28 с.

[2] Информационная технология. Криптографическая защита информации. Блочные шифры : ГОСТ Р 34.12-2015. – Введ. 01.01.2016. – М.: Стандартинформ, 2015. – 16 с.

[3] Smyshlyaev, S. Cryptographic Algorithms for GOST : RFC 7836 [Электронный ресурс] / S. Smyshlyaev [и др.]. – Режим доступа: https://datatracker.ietf.org/doc/html/rfc7836. – Дата доступа: 27.09.2026.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Базовый блочный шифр

```
package gost

import (
  "encoding/binary"
  "math/bits"
)

const (
  BlockSize = 8
  KeySize   = 32
)

// id-tc26-gost-28147-param-Z (RFC 7836)
var sbox = [8][16]uint8{
   {0xc, 0x4, 0x6, 0x2, 0xa, 0x5, 0xb, 0x9, 0xe, 0x8, 0xd, 0x7, 0x0, 0x3,
0xf, 0x1},
   {0x6, 0x8, 0x2, 0x3, 0x9, 0xa, 0x5, 0xc, 0x1, 0xe, 0x4, 0x7, 0xb, 0xd,
0x0, 0xf},
   {0xb, 0x3, 0x5, 0x8, 0x2, 0xf, 0xa, 0xd, 0xe, 0x1, 0x7, 0x4, 0xc, 0x9,
0x6, 0x0},
   {0xc, 0x8, 0x2, 0x1, 0xd, 0x4, 0xf, 0x6, 0x7, 0x0, 0xa, 0x5, 0x3, 0xe,
0x9, 0xb},
   {0x7, 0xf, 0x5, 0xa, 0x8, 0x1, 0x6, 0xd, 0x0, 0x9, 0x3, 0xe, 0xb, 0x4,
0x2, 0xc},
   {0x5, 0xd, 0xf, 0x6, 0x9, 0x2, 0xc, 0xa, 0xb, 0x7, 0x8, 0x1, 0x4, 0x3,
0xe, 0x0},
   {0x8, 0xe, 0x2, 0x5, 0x6, 0x9, 0x1, 0xc, 0xf, 0x4, 0xb, 0x0, 0xd, 0xa,
0x3, 0x7},
   {0x1, 0x7, 0xe, 0xd, 0x0, 0x5, 0x8, 0x3, 0x4, 0xf, 0xa, 0x6, 0x9, 0xc,
0xb, 0x2},
}

func KeysForRoundEncrypt(key []byte, rounds int) []uint32 {
  subkeys := make([]uint32, 8)
  for i := range 8 {
    subkeys[i] = binary.LittleEndian.Uint32(key[i*4 : i*4+4])
  }

  result := make([]uint32, rounds)
  for i := range rounds {
    if i < 24 {
      result[i] = subkeys[i%8]
    } else {
      result[i] = subkeys[7-i%8]
    }
  }

  return result
}

func KeysForRoundDecrypt(key []byte, rounds int) []uint32 {
  subkeys := make([]uint32, 8)
  for i := range 8 {
    subkeys[i] = binary.LittleEndian.Uint32(key[i*4 : i*4+4])
  }
```

```
  result := make([]uint32, rounds)
  for i := range rounds {
    if i < 8 {
      result[i] = subkeys[i]
    } else {
      result[i] = subkeys[7-i%8]
    }
  }

  return result
}

func f(left uint32, subkey uint32) uint32 {
  s := left + subkey

  var result uint32
  for i := range 8 {
    // 4 bits
    x := uint8((s >> (4 * i)) & 0xF)
    replaced := sbox[i][x]

    result |= uint32(replaced) << (4 * i)
  }

  result = bits.RotateLeft32(result, 11)
  return result
}

func BlockEncryptParts(input []byte, key []byte, rounds int) (uint32, uint32) {
  left := binary.LittleEndian.Uint32(input[0:4])
  right := binary.LittleEndian.Uint32(input[4:8])

  subkeys := KeysForRoundEncrypt(key, rounds)

  for i := range rounds {
    left, right = right^f(left, subkeys[i]), left
  }

  return right, left
}

func BlockDecryptParts(input []byte, key []byte, rounds int) (uint32, uint32) {
  left := binary.LittleEndian.Uint32(input[0:4])
  right := binary.LittleEndian.Uint32(input[4:8])

  subkeys := KeysForRoundDecrypt(key, rounds)

  for i := range rounds {
    left, right = right^f(left, subkeys[i]), left
  }

  return right, left
}

func BlockEncrypt32(input []byte, key []byte) []byte {
  left, right := BlockEncryptParts(input, key, 32)

  combined := make([]byte, 8)
  binary.LittleEndian.PutUint32(combined[0:4], left)
  binary.LittleEndian.PutUint32(combined[4:8], right)
  return combined
}
```

```
func BlockDecrypt32(input []byte, key []byte) []byte {
  left, right := BlockDecryptParts(input, key, 32)

  combined := make([]byte, 8)
  binary.LittleEndian.PutUint32(combined[0:4], left)
  binary.LittleEndian.PutUint32(combined[4:8], right)
  return combined
}
```

Листинг А.2 – Режим простой замены

```
package gost

func ECBEncrypt(data, key []byte) []byte {
  padded := pad(data)
  result := make([]byte, len(padded))
  for i := 0; i < len(padded); i += 8 {
    copy(result[i:i+8], BlockEncrypt32(padded[i:i+8], key))
  }

  return result
}

func ECBDecrypt(data, key []byte) []byte {
  result := make([]byte, len(data))
  for i := 0; i < len(result); i += 8 {
    copy(result[i:i+8], BlockDecrypt32(data[i:i+8], key))
  }

  return unpad(result)
}
```

Листинг А.3 – Режим гаммирования

```
package gost

import (
  "crypto/rand"
  "encoding/binary"
)

func Gamma(iv []byte, key []byte, length int) []byte {
  if len(iv) != 8 {
    panic("iv must be 8 byte")
  }

  n3, n4 := BlockEncryptParts(iv, key, 32)

  gamma := make([]byte, 0, length)
  for len(gamma) < length {
    n3 = n3 + 0x01010101 // mod 2^32

    // mod 2^32-1
    n4 = n4 + 0x01010104
    if n4 < 0x01010104 { // overflow
      n4++
    }
    if n4 == 0xFFFFFFFF {
      n4 = 0
    }

    block := make([]byte, 8)
    binary.LittleEndian.PutUint32(block[0:4], n3)
```

```
    binary.LittleEndian.PutUint32(block[4:8], n4)

    enc := BlockEncrypt32(block, key)
    gamma = append(gamma, enc...)
  }

  return gamma[:length]
}

func GostGammaEncrypt(data, key []byte) ([]byte, []byte) {
  iv := make([]byte, 8)
  if _, err := rand.Read(iv); err != nil {
    panic(err)
  }

  gamma := Gamma(iv, key, len(data))

  cypherText := make([]byte, len(data))
  for i := range data {
    cypherText[i] = data[i] ^ gamma[i]
  }

  return cypherText, iv
}

func GostGammaDecrypt(data, key, iv []byte) []byte {
  gamma := Gamma(iv, key, len(data))

  result := make([]byte, len(data))
  for i := range data {
    result[i] = data[i] ^ gamma[i]
  }

  return result
}
```

Листинг А.4 – Режим гаммирования с обратной связью

```
package gost

import "crypto/rand"

func GammaWithFeedbackEncrypt(data, key []byte) ([]byte, []byte) {
  iv := make([]byte, 8)
  if _, err := rand.Read(iv); err != nil {
    panic(err)
  }

  padded := pad(data)
  cipherText := make([]byte, len(padded))
  prev := iv

  for i := 0; i < len(padded); i += 8 {
    gamma := Gamma(prev, key, 8)

    block := padded[i : i+8]
    for j := range block {
      cipherText[i+j] = block[j] ^ gamma[j]
    }

    prev = cipherText[i : i+8]
  }
```

```
  return cipherText, iv
}

func GammaWithFeedbackDecrypt(data, key, iv []byte) []byte {
  result := make([]byte, len(data))
  prev := iv

  for i := 0; i < len(data); i += 8 {
    gamma := Gamma(prev, key, 8)

    block := data[i : i+8]
    for j := range block {
      result[i+j] = block[j] ^ gamma[j]
    }

    prev = data[i : i+8]
  }

  return unpad(result)
}
```

Листинг А.5 – Вычисление кода аутентичности

```
package gost

import (
  "bytes"
  "encoding/binary"
)

func parts(block, key []byte, rounds int) (uint32, uint32) {
  second, first := BlockEncryptParts(block, key, rounds)
  return first, second
}

func MACProduce(data, key []byte, l int) []byte {
  if l <= 0 || l > 32 {
    panic("l can't be less than 1 or more than 32")
  }
  padded := zeroPad(data)

  n1, n2 := parts(padded, key, 16)

  for i := 8; i < len(padded); i += 8 {
    w1 := binary.LittleEndian.Uint32(padded[i : i+4])
    w2 := binary.LittleEndian.Uint32(padded[i+4 : i+8])

    n1 ^= w1
    n2 ^= w2

    combined := make([]byte, 8)
    binary.LittleEndian.PutUint32(combined[0:4], n1)
    binary.LittleEndian.PutUint32(combined[4:8], n2)

    n1, n2 = parts(combined, key, 16)
  }

  byteLen := (l + 7) / 8

  result := make([]byte, 4)
  binary.BigEndian.PutUint32(result, n1)
  return result[:byteLen]
}
```

```
func GammaEncryptWithMAC(data, key []byte) ([]byte, []byte, []byte) {
  cypherText, iv := GostGammaEncrypt(data, key)

  mac := MACProduce(data, key, 32)

  return cypherText, iv, mac
}

func GammaDecryptWithMAC(data, key, iv, mac []byte) []byte {
  decrypted := GostGammaDecrypt(data, key, iv)

  decryptedMac := MACProduce(decrypted, key, 32)

  if !bytes.Equal(mac, decryptedMac) {
    panic("cypherText was corrupted")
  }

  return decrypted
}
```